RED EDEN · دليل تركيب

تركيب أنظمة اللاعب على Unity Third Person

المخزون، التفاعل، قواعد البيانات الجاهزة، الـ Prefabs، والصومعة الزراعية

> **الهدف:** نقل أنظمة RED EDEN إلى شخصية جديدة مبنية على Unity Starter Assets — Third Person Controller، من دون استبدال حركة اللاعب أو الكاميرا الأصلية.

إصدار المشروع المرجعي: `Unity 6000.3.16f1`

الحزمة الأساسية تعمل دون ARES. المساعد الصوتي/الذكي وحدة اختيارية موضحة في نهاية الدليل.

يُفضّل تنفيذ الخطوات أولًا على نسخة احتياطية أو فرع تجريبي من المشروع.

١. ما الذي ستحتاجه؟

- مشروع Unity يحتوي Unity Starter Assets — Third Person Controller، مع شخصية ومكوّنات الحركة والكاميرا الأصلية.
- حزمة `RedEden-Core.unitypackage`.
- أو مجلدا `Assets/RED EDEN` و`Assets/Prefabs` من المستودع، مع ملفات `.meta` كما هي.
- أنظمة Devion Inventory System وTriggers وUtilities وUI Widgets المتوافقة. إذا كانت Devion مثبتة مسبقًا، استخدم نسختك الحالية ولا تستورد نسخًا مكررة منها.
- حزمة Unity Input System وتفعيلها من `Project Settings > Player > Active Input Handling`.

> لا تستورد ولا تضف `Devion Third Person Controller`. احتفظ بمكوّنات Unity Starter Assets مثل `ThirdPersonController` و`PlayerInput` و`CharacterController` والكاميرا كما هي.

ترتيب الاستيراد

1. افتح نسخة احتياطية من المشروع المستهدف. الطريقة الأسهل لمشروع نظيف هي استيراد `RedEden-Core.unitypackage` مرة واحدة؛ تحتوي على الأنظمة والـ Prefabs واعتماديات Devion اللازمة.
2. بدلًا من الحزمة الكاملة، يمكن نسخ مجلدي `Assets/RED EDEN` و`Assets/Prefabs` إلى `Assets/` في المشروع مع الحفاظ على المسارات وكل ملفات `.meta`. لا تضعهما داخل `Assets/Assets`. إذا غابت Devion، استورد `RedEden-Devion-Dependencies.unitypackage` فقط؛ لا تكرر Devion المثبتة.
3. لا تستخدم النسخ اليدوي والحزمة الكاملة معًا في تركيب واحد. لا تستورد `RedEden-ARES-Optional.unitypackage` الآن إلا إذا كنت تحتاج وظائف ARES.
4. انتظر اكتمال استيراد الأصول وتجميع السكربتات، ثم تحقق من عدم وجود أخطاء في نافذة `Console`.

إعدادات المشروع

اجعل `Active Input Handling` على `Input System Package (New)`، ثم أعد تشغيل المحرر إذا طلب Unity ذلك. استخدم إعدادات التصادم الافتراضية المناسبة للعبة وتأكد من أن طبقات اللاعب ومحفزات التفاعل لا تمنع بعضها.

٢. إعداد اللاعب الجديد

1. استخدم شخصية Unity Third Person الموجودة في المشروع أو أنشئها من Starter Assets. لا تستبدل سكربتات الحركة والكاميرا.
2. عيّن Tag اللاعب إلى `Player`؛ يجب أن يكون الاسم مطابقًا تمامًا.
3. احتفظ بـ `CharacterController` ومكوّنات الإدخال والحركة الموجودة. لا تضف Devion Character Controller.
4. أضف `InventoryManager` واحدًا إلى مشهد اللعب. يمكن أن يكون على كائن إدارة مستقل بدل كائن اللاعب.
5. أضف واجهة `Inventory Window` من تثبيت Devion المرخّص لديك، واربطها وفق إعدادات Devion. الواجهة نفسها غير مضمنة في حزمة RED EDEN. يلزم وجود `EventSystem` للواجهات التي تعتمد على النقر.

> التفاعل من Devion يعتمد على تعرّف `InventoryManager.PlayerInfo` إلى اللاعب عبر Tag. إذا لم يكن Tag مضبوطًا، فلن تعمل حدود التفاعل كما هو متوقع.

قاعدة البيانات الصحيحة

1. في حقل قاعدة البيانات الرئيسية على `InventoryManager`، عيّن `RED EDEN Core Inventory Database.asset`.
2. في قائمة قواعد البيانات الفرعية، أضف `RED EDEN Item Database.asset`.
3. اترك قواعد بيانات العناصر الأخرى المطلوبة للعبة ضمن القائمة الفرعية ولا تكرر قاعدة RED EDEN.

> لا تجعل `RED EDEN Item Database.asset` القاعدة الرئيسية: فهي تحتوي عناصر اللعبة، لكن قائمة إعدادات Devion فيها فارغة. القاعدة الأساسية الجديدة تحتوي إعدادات Default وInput وUI وNotifications وSaving/Loading اللازمة للنظام.

يمكن الاحتفاظ بأكثر من قاعدة عناصر فرعية. اجعل لكل عنصر معرفًا فريدًا، وتأكد أن عناصر الوصفات وعناصر المخزون تشير إلى الأصول نفسها.

المستودع يتضمن ملفات قواعد البيانات الثلاثة كاملة داخل `Assets/RED EDEN/Crafting/Data/`: قاعدة Core وإعدادات Devion، وقاعدة عناصر RED EDEN، وقاعدة التصنيع والموارد والوصفات. لا تستخدم قاعدة العناصر وحدها كبديل للقاعدة الأساسية.

٣. التفاعل والمخزون

المخزون

- اختبر فتح نافذة المخزون وإغلاقها وإضافة عنصر تجريبي قبل تركيب المحطات.
- عناصر RED EDEN مسجلة في قاعدة العناصر الفرعية؛ المخزون الفعلي يظل تابعًا لـ Devion.
- إذا كانت النافذة فارغة، راجع قواعد البيانات المعيّنة في `InventoryManager` وتأكد من وجود العنصر المطلوب في قاعدة فرعية نشطة.

محطات التصنيع وعقد الموارد

1. أضف `RedEdenCraftingStation` إلى محطة الطباعة، وعيّن `RED EDEN Crafting Database.asset`.
2. أضف Collider للمحطة واترك نطاق التفاعل الذي ينشئه Devion أو اضبط `useDistance` من الـ Inspector.
3. أضف `MartianResourceNode` إلى عقدة المورد، ثم عيّن تعريف المورد والكمية المتبقية.
4. تفاعل بالاقتراب والضغط على **E**. اضبط Tag اللاعب إلى `Player` وتأكد أن التصادمات تسمح بوصول أحداث الـ Trigger.

الصومعة الزراعية

اسحب الـ prefab الجاهز `Hydroponic Farm M01 - Agricultural Silo.prefab` من `Assets/RED EDEN/Crafting/Data/Prefabs/Machines/` إلى المشهد. يتضمن المستودع ملف `Hydroponic_Farm_M01.fbx` ونسخة النموذج التي يعتمد عليها الـ prefab، إضافة إلى مكونات `HydroponicFarm` وواجهة الشاشة ومراجع عناصر الأكسجين والمحاصيل. لا يلزم تشغيل أداة إنشاء prefab أثناء التركيب المعتاد.

تأكد أن `InventoryManager` يستخدم القاعدة الأساسية والفرعية الموضحتين أعلاه. اضبط أوقات الإنتاج وكميات المحاصيل والأكسجين وحدود التخزين ونوع المحصول من الـ Inspector.

- **E:** جمع الإنتاج المخزن.
- **F:** تبديل المحصول إذا فُعّل الخيار في إعدادات الصومعة.

> استخدم الـ prefab الجاهز ومراجع النموذج المرفقة كما هي. إذا ظهر Missing Model أو Missing Script، تحقق أولًا من نسخ مجلدات `Assets/RED EDEN` و`Assets/Prefabs` كاملة مع ملفات `.meta`، ومن تثبيت اعتماديات Devion المطلوبة.

٤. الحفظ والاختبار واستكشاف الأخطاء

الحفظ

يعتمد المخزون على آلية الحفظ في Devion. تحفظ محطات RED EDEN وعقد الموارد والصومعة حالتها ومعرّفاتها الثابتة ضمن مسار حفظ المشهد. عند نسخ محطة أو عقدة، أنشئ معرّفًا ثابتًا جديدًا لكل نسخة قبل الاختبار.

اختبار التركيب

1. شغّل المشهد وتأكد أن الشخصية تتحرك كما كانت قبل إضافة الأنظمة.
2. افتح واجهة المخزون وتأكد من ظهور عناصر RED EDEN عند إضافتها.
3. اقترب من محطة تصنيع واضغط E؛ جرّب وصفة بمواد كافية وأخرى غير متاحة.
4. اجمع موردًا من عقدة، ثم احفظ وأعد تحميل المشهد للتحقق من الكمية.
5. راقب الصومعة حتى تكتمل دورة إنتاج، واجمع المحصول أو الأكسجين.
6. شغّل اختبارات `RedEden.Crafting` من `Window > General > Test Runner`.

العَرَض	ما يجب فحصه
لا يظهر اللاعب داخل نطاق التفاعل	Tag = Player، وجود Collider، InventoryManager ونطاق التفاعل، وطبقات التصادم.
المخزون أو الإعدادات لا تعمل	القاعدة الرئيسية هي Core Inventory Database، وقاعدة RED EDEN ضمن القواعد الفرعية.
التفاعل لا يستجيب للمفتاح	Input System مفعّل؛ راجع مفتاح التفاعل المحدد على المحطة وإعدادات Devion.
لا تظهر الصومعة بالشكل المتوقع	تحقق من وجود نموذج FBX وملفات .meta، ثم افتح الـ prefab الجاهز وتحقق من مراجع النموذج والخامات.

لا تغيّر نظام الحركة لتجاوز مشكلة في المخزون أو التفاعل؛ افحص أولًا Tag وقاعدة البيانات وإعداد الإدخال.

٥. ARES الاختيارية

ARES ليست شرطًا للمخزون أو التفاعل أو التصنيع. عند نسخ مجلد `Assets/RED EDEN` كاملًا إلى المشروع، ثبّت اعتماديات ARES أدناه قبل تجميع سكربتاتها، أو استبعد مجلد `Assets/RED EDEN/ARES/` إذا لم تكن تحتاجها. استورد `RedEden-ARES-Optional.unitypackage` لإضافة موارد ARES واعتماد Devion Stat System عند استخدام الوحدة.

اعتماديات إضافية

- حزمة Unity `com.whisper.unity` من Package Manager باستخدام Git URL:
  `https://github.com/Macoron/whisper.unity.git?path=/Packages/com.whisper.unity`.
- خدمة Ollama محلية ونموذج مناسب؛ إعداد الخادم والعنوان والنموذج من ملفات إعداد ARES.
- إذن الميكروفون عند تفعيل إدخال الصوت، وإعدادات إخراج الصوت عند استخدام الرد الصوتي.

إعداد الشخصية

1. أضف مكونات ARES المطلوبة على كائن مستقل أو اللاعب، واربط ملفات الإعدادات والموفرات من الـ Inspector.
2. يمكن إضافة `ARESPlayerVitals` إلى الشخصية. عند عدم ربط `StatsHandler` من Devion، يستخدم النظام قيم الحالة الاحتياطية؛ لا يغيّر هذا حركة Unity Third Person.
3. مفتاح تفاعل ARES في الإعداد الحالي هو **V**، بينما تفاعل محطات RED EDEN هو **E**.
4. اختبر ARES اختياريًا بعد التأكد من تشغيل الخادم المحلي والاعتماديات. لا تعتبر إعداد المساعد ناجحًا لمجرد نجاح بناء حزمة Core.

> أداة إعداد ARES الموجودة في المشروع الأصلي مرتبطة بمشهد الاختبار و`StatsHandler` الخاص بـ Devion. لا تستخدمها على الشخصية الجديدة دون إعادة ربطها يدويًا. إضافة الصوت قد تتطلب تنزيل النموذج عند التشغيل.

ترخيص الأصول

تتضمن الحزم ملفات Devion اللازمة للتكامل. يجب أن يملك كل من يستوردها صلاحية استخدام هذه الملفات وفق ترخيص Devion. لا تعِد نشر ملفات الطرف الثالث خارج نطاق الإذن الممنوح.